

LoanLens — Phase 7 Extension

Architecture, System Flow, Results, Deployment & Viva Preparation

This section is intended to be appended to the existing Final Project Report.

Phase 7.1 — Project Architecture

LoanLens follows a dual-runtime architecture. The Node.js/Express application handles server-side rendering and acts as the API gateway, while Python Flask handles Machine Learning inference.

Layer	Technology	Responsibility
Presentation	EJS + HTML + CSS + Vanilla JavaScript	UI interface, forms, charts and result presentation
Application/API	Node.js + Express + TypeScript	SSR, routing, API gateway and frontend services
ML Inference	Python + Flask	Preprocessing, model loading and prediction
ML Models	scikit-learn + NumPy	Four production classifiers + scratch Logistic Regression
Deployment	Docker + Render	Unified production container and cloud hosting

Architecture flow: Browser → Node.js/Express → Flask Inference API → Preprocessor → Selected ML Model → Prediction Result → Node.js → Browser

Phase 7.2 — ML Workflow

- Load the Loan Default dataset.
- Validate missing values and duplicate rows.
- Remove LoanID identifier.
- Separate predictors and Default target.
- Perform an 80/20 stratified train/test split.
- Standardize the 9 numerical features.
- One-hot encode the 7 categorical features.
- Generate the final 24-feature matrix.
- Train Logistic Regression, KNN, Naive Bayes and Decision Tree.
- Implement Logistic Regression independently using NumPy.
- Evaluate the models using Accuracy, Precision, Recall, F1-score and ROC-AUC.
- Perform cross-validation and hyperparameter tuning.
- Store production preprocessing and model artifacts.
- Serve predictions through Flask and integrate them with LoanLens.

Phase 7.3 — System Data Flow

Step	Component	Action
1	User	Enters borrower information
2	Frontend	Validates and submits assessment
3	Node/Express	Receives request and forwards inference request
4	Flask	Receives structured borrower data

Step	Component	Action
5	Preprocessor	Applies scaling and categorical encoding
6	ML Model	Calculates default probability and class
7	Flask	Returns prediction payload
8	Node/Express	Returns/serves result to frontend
9	Frontend	Displays probability, classification and risk information

Phase 7.4 — Final Model Results

The final untouched test set contains 51,070 observations. The following benchmark is the documented production comparison.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.6764	0.2196	0.6995	0.3342	0.7532
KNN	0.8851	0.5455	0.0070	0.0137	0.6927
Naive Bayes	0.8849	0.6080	0.0256	0.0492	0.7499
Decision Tree	0.8849	0.5888	0.0302	0.0574	0.7254

The target is imbalanced, so the high Accuracy values of KNN, Naive Bayes and Decision Tree should be interpreted together with their very low Default Recall. The production benchmark records Logistic Regression with the highest ROC-AUC and substantially higher Default Recall among these four models.

Phase 7.5 — Cross-Validation Results

Model	5-Fold CV ROC-AUC	Variation
Logistic Regression	0.7426	± 0.0063
KNN	0.6753	± 0.0068
Naive Bayes	0.7396	± 0.0044
Decision Tree	0.7036	± 0.0057

Cross-validation was used to examine model performance across multiple folds instead of relying only on one train/test split.

Phase 7.6 — Scratch Logistic Regression Viva Explanation

A simple viva explanation:

"I implemented Logistic Regression from scratch using NumPy. First, I initialized the weights and bias. Then I calculated the linear combination, applied the sigmoid function to obtain probabilities, calculated Binary Cross-Entropy loss, calculated gradients, and updated the parameters using Gradient Descent. Finally, I used the learned parameters to generate probabilities and class predictions."

Important terms to remember:

- Sigmoid: converts a real-valued score into a probability between 0 and 1.
- Binary Cross-Entropy: measures the difference between predicted probabilities and binary targets.
- Gradient: indicates the direction and magnitude used to update model parameters.
- Gradient Descent: iteratively updates parameters to minimize the loss.
- Learning rate: controls the size of each parameter update.

Phase 7.7 — Deployment Architecture

The production deployment uses a unified Docker container containing both Node.js and Python runtimes. Node/Express is exposed through the Render-provided public port, while Flask runs internally on 127.0.0.1:5001.

Component	Production role
Render	Cloud hosting platform
Docker	Packages Node.js + Python + application artifacts
Node.js/Express	Public web server and API gateway
EJS	Server-side rendered pages
Flask	Internal ML inference engine
scikit-learn 1.6.1	Runtime aligned with serialized model artifacts
GitHub	Source-code repository

Live application: <https://loanlens-hynm.onrender.com>

Phase 7.8 — Production Verification Checklist

Check	Status
Node production startup	PASS
Flask startup	PASS
ML artifacts loaded	PASS
Express → Flask communication	PASS
GET /api/health	200 OK
GET /api/models	200 OK
GET /api/models/comparison	200 OK
GET /api/data/insights	200 OK
POST /api/predict	200 OK
Frontend SSR pages	PASS
Render service live	PASS

Phase 7.9 — Common Viva Questions & Short Answers

Question	Answer
What is the problem type?	Binary classification: predict Default (1) or No Default (0).
Why is Accuracy not enough?	The target is imbalanced, so Accuracy can hide poor detection of Default cases.

Question	Answer
Why use Logistic Regression?	It directly models binary-class probabilities and provides interpretable probability output.
Why scale numerical features?	Scaling puts numerical variables on comparable scales and is important for scale-sensitive models.
Why encode categorical features?	ML estimators require numerical representations of categorical inputs.
Why remove LoanID?	It is an identifier and does not represent a meaningful predictive borrower attribute.
What is ROC-AUC?	A threshold-independent measure of how well the model separates the two classes across all thresholds.
What did you implement from scratch?	Logistic Regression using NumPy and Gradient Descent.
Why use cross-validation?	To evaluate performance across multiple folds and assess stability.
Where is Flask used?	As the Python ML inference service that loads preprocessing and model artifacts and serves predictions.
How does frontend communicate with ML?	The frontend submits to the Node/Express application, which routes inference to Flask.
Where is the application deployed?	Render, using a unified Docker container.

Phase 7.10 — Final Viva Checklist

- Know the problem statement in one or two sentences.
- Know the dataset size: 255,347 rows and 18 columns.
- Know the target: Default, where 0 = No Default and 1 = Default.
- Know why the target imbalance matters.
- Know the 9 numerical and 7 categorical features.
- Know the preprocessing: StandardScaler + OneHotEncoder.
- Know the final 24 encoded features.
- Know all four algorithms.
- Know the scratch Logistic Regression steps.
- Know Accuracy, Precision, Recall, F1-score and ROC-AUC definitions.
- Know the final benchmark values.
- Know the production Logistic Regression configuration.
- Know the Flask and Node/Express roles.
- Know the deployment architecture.
- Be able to explain the end-to-end prediction flow.

Phase 7.11 — Final Project Statement

LoanLens is a complete end-to-end Machine Learning application for educational loan-default prediction. It combines structured-data preprocessing, four classification models, a scratch Logistic Regression implementation, evaluation and cross-validation, visualization, Flask inference, Node.js/Express integration, a server-rendered frontend, Docker packaging, and Render deployment.

Final status: Project implementation, deployment, API verification and report documentation are complete. The remaining activity is academic demonstration/viva preparation.